

클로드코드 Phase 2

사용자

미니 노션 — 한 권 요약본

탐처럼, 통제하며, 지키면서, 세상에 내보내며 만들기 · Phase 2 (Day 6-16) 압축판

이 요약본은 Day 6~16 열한 권을 **핵심만 남겨 한 편으로 압축한** 것입니다. 자세한 예시·화면·실습은 각 [book-dayNN.md](#) 를 보세요.

이 책을 한 문단으로

Phase 1이 "혼자서 만들 수 있다"였다면, Phase 2는 **기획(PM-Skills) → 디자인(클로드 디자인) → 개발(하네스 엔지니어링)**을 핸드오프로 잇는 역할 분리형으로 미니 노션 웹서비스를 짓는다. 빼대는 **DB 설계(정규화·UUID)**와 **LLM 엔지니어링 4단계(프롬프트→컨텍스트→하네스→루프)**. 여기에 **외부 API(CORS·프록시)**, **인증·인가와 암호화**, **구글로그인(JWT)**, **혹과 Git 워크트리**, **이미지 스토리지 분리 저장**, 그리고 **Git·Vercel 배포와 모니터링**까지 얹는다. 관통하는 한 문장: **모델의 선의에 의존하지 말고, 구조로 강제하라.**

미니 노션 제작 파이프라인 — 기획(PRD) → 디자인([DESIGN.md](#)) → DB(정규화) → 개발(TDD·SDD) → API(CORS·프록시) → 로그인(인증·인가·OAuth) → 안전장치(혹) → 병렬구현(워크트리) → 이미지(Storage) → 배포·모니터링(Vercel)

Week 2 (Day 6-10) — 기획 → 디자인 → 개발을 핸드오프로 잇기

Day 6 · 실무형 협업 구조와 기획 (PM)

- **역할 통합형 vs 역할 분리형**: 통합형=빠름(but 일관성·유지보수 약함), **분리형=디자인 일관성·유지보수 강함** → 실무 선택.
- **역할별 AI 도구**: 기획=**PM-Skills**, 디자인=**클로드 디자인**, 개발=**클로드 코드 + Supabase**(TDD/SDD 하네스). 지라·피그마를 직접 안 배워도 AI가 대체.
- **핸드오프**: 뒤로 갈수록 충실도↑ (PRD → 로-파이 → 하이-파이 → 개발). **AI 시대엔 로-파이(와이어프레임) 생략** — PRD에서 바로 하이-파이로. (로-파이=UX, 하이-파이=UI)
- **PM 3역할**: 기획/인터뷰 → 프로젝트 관리 → 데이터 분석. (Product Manager=사업성·고객 / Project Manager=일정)
- **기획/인터뷰 7단계**(👉 수동 ↔ ⚡ 명령 번갈아): 유저 인터뷰 → 요약(/summarize-meeting) → 인사이트·전략가정(👉) → 가정 공격(/strategy-red-team) → 우선순위(/prioritization-frameworks , 필수·선택·확장) → 브리프(👉) → PRD(/create-prd). 앞 산출물을 @ 로 이어받는 파일 파이프라인.

한 문장 — 실무는 역할 분리형이다. 각 역할을 AI 도구로 잇고, 기획은 7단계로 유저의 숨은 니즈를 찾아 PRD로 마무리한다.

Day 7 · 클로드 디자인 — 디자인 시스템과 하이파이

- **디자인 프로세스:** 기초 = 레퍼런스 조사 → 디자인 시스템 → 화면 디자인 / 심화=중간에 **메인페이지 시안** 2~3개 추가.
- **디자인 시스템** = 규칙(컬러 파운데이션 · 타이포그래피 · 컴포넌트), **화면 디자인** = 부품 조립.
- **레퍼런스 도구:** UI볼·WWIT(국내), mobbin(해외).
- **참고 샘플 3종:** 사진(스크린샷) · 피그마(.fig) · 소스코드(+**DESIGN.md**). **DESIGN.md** 모음=켓디자인MD.
- **폰트 저작권:** 상업적+무료 → 추천 **프리텐다드**. 결과물(컬러·타이포·컴포넌트)이 다음 단계로 핸드오프.

한 문장 — 화면(부품 조립)에 들어가기 전, 디자인 시스템(규칙: 컬러·타이포·컴포넌트)부터 세운다.

Day 8 · 디자인을 코드로 넘기고 DB를 설계하기

- **DESIGN.md** = 새 화면을 추가해도 톤앤매너·컬러 일관성을 지키는 디자인 가이드(단일 원본).
- **메타프롬프트** = "결과물"이 아니라 "**결과물을 만들 프롬프트**"를 먼저 만들기(반드시 지킬 조건·피해야 할 예시·완료 기준). 잘 만든 프롬프트 1개를 재사용.
- **CLAUDE.md** 등록: 클로드 코드는 매 채팅마다 CLAUDE.md를 먼저 읽음 → "디자인 작업 전 **DESIGN.md** 먼저 읽어라"를 못 박으면 **확률적** → **결정론적**.
- **DB:** NoSQL(문서: 컬렉션>다큐먼트>필드, 자유 / Firebase) vs SQL(표: 테이블>행/열, 안정 / Supabase=PostgreSQL).
- **정규화** = 중복을 별도 테이블로 분리 → 변경 1회로 끝. **관계 3유형** 1:1 · 1:N · N:M(→ 중간 테이블로 두 개의 1:N 분해).
- **PK**(기본키, 행 식별) · **FK**(외래키, 다른 테이블 PK 참조) · **Join**(PK-FK로 재결합) · **ERD**(설계도, ERDCloud).

한 문장 — 디자인은 DESIGN.md+CLAUDE.md로 강제하고, 데이터는 정규화(PK/FK)로 중복을 없앤다.

Day 9 · DB 실전(Supabase)과 LLM 엔지니어링 4단계

- **정규화 실전:** 큰 Page 테이블 → User 분리. Supabase 기본 제공 **auth.users**(UID=UUID), 서비스용은 **public.Profile**로 분리.
- **UUID vs auto_increment:** auto_increment=속도↑·예측 가능(위험), UUID=보안↑. ⚠️ 쿠팡 3370만 건 유출 — PK를 순차 정수로 뒤서 숫자만 훑어 뚫림 → 식별자는 **UUID**.
- **LLM 엔지니어링 4단계:** ①프롬프트(잘 부탁) → ②컨텍스트(+자료·경로 제공) → ③하네스(+안전장치) →

④루프(+자동화). (AI 성능 = 모델 + 도구 제어)

- **TDD**(테스트 먼저)/**SDD**(명세 먼저). 테스트코드 핵심 = **회귀 방지**(새 기능이 기존 기능 깨는지 자동 검증). SDD 효과: **6,715줄 → 399줄**.

한 문장 — 식별자는 **UUID**로(쿠팡 반면교사), AI는 **하네스(TDD·SDD)**로 통제하며, 명세 먼저·구현은 그대로.

Day 10 · 하네스 플러그인과 API·JSON

- 하네스 도구: 내장 스킴(`/plan` · `/simplify` · `/code-review` · `/verify`) + 외부 플러그인 **Superpowers(TDD)** · **Speckit(SDD)**.
- **TDD+SDD 전개도**: 명세 작성(`/specify`) → 재검토 2회(`/clarify` , ★가장 중요) → 설계도(`/plan`) → 작업목록(`/tasks`) → 구현(`/implement`). 설계 후 **컨텍스트 초기화**하고 **저렴한 모델로 구현**.
- **API 엔드포인트**: 프론트(`naver.com` , HTML) vs **백엔드**(`.../posts/1` , **JSON**) — 실무 API=백엔드. **GET**(조회) vs **POST·PATCH·DELETE**(등록·수정·삭제).
- API 종류: 백엔드 / 오픈(퍼블릭) / 공공(활용신청). 데이터 표현은 **JSON**(XML보다 간결).
- **Postman**: 브라우저 주소창은 GET만 → POST/PATCH/DELETE 실습은 포스트맨. **HTTP 상태코드**: 200(성공)·400(클라이언트 잘못)·500(서버 잘못).

한 문장 — Superpowers·Speckit로 명세→설계→구현을 자동화하고, 실무 API(백엔드·JSON)를 상태코드로 읽는다.

Week 3+ (Day 11-16) — 연결하고, 지키고, 세상에 내보내기

Day 11 · 스켈레톤 UI와 CORS·프록시

- **스피너 vs 스켈레톤 UI**: 콘텐츠 자리를 회색 블록으로 미리 그리는 **스켈레톤 권장**(화면 덜 튼).
- **SOP**(동일 출처 정책): 브라우저는 **같은 출처의 API만** 호출 가능. **CORS**: 서버가 "이 출처 허용"을 선언하는 예외.
- **Origin**은 브라우저에만 해당 — 서버·모바일 앱은 CORS 검사 안 받음(→ 프록시 해법의 근거).
- 차단 방식: 간단한 요청=응답은 오나 **열람 금지**, 복잡한 요청=**사전요청(preflight)** 거절 시 실제요청 안 보냄.
- **프록시 서버**: 브라우저 → 내 서버(프록시) → 외부 API → 서버 간 통신은 CORS 무관. 선택지: **프론트엔드 서버 프록시** / **Supabase Edge Function**.
- ⚠ `net::ERR_FAILED 200 (OK)` — 서버는 정상인데 브라우저가 막은 것(유료·서버다운 아님).

한 문장 — CORS는 브라우저가 막는 것. 서버를 못 고치면 프록시로 우회한다.

Day 12 · 보안 기초 — 인증·인가, 해시, 세션 vs 토큰

- 회원가입=회원 테이블에 1줄 등록, 로그인=이미 등록된 회원임을 증명.
- 인증(Authentication)=로그인 증표 받기(1회) / 인가(Authorization)=요청마다 증표로 증명하기. 증표는 header, 데이터는 body.
- 양방향 암호화(복호화 O)=개인정보(화면에 다시 표시) / 단방향 해시(복호화 X)=비밀번호(예: bcrypt). → 비밀번호는 찾기 불가, 재발급.
- 레인보우 테이블(해시값 역추적) 방어 = SALT(해시 전 무작위 문자열).
- 세션 방식(세션 테이블 저장, 중복로그인 제한·강제 로그아웃 가능) vs 토큰 방식(JWT)(토큰 안에 로그인여부 +만료, 세션 테이블 불필요·효율). 설계 순서: 토큰(무료)으로 시작 → 필요 시 세션 추가.
- 소셜로그인/OAuth: 기존 가입 서비스로 자동 로그인(가입 이탈 방지). 흐름: 비밀 CODE → 구글 전용 토큰 → Supabase 전용 토큰(JWT)(이후 인가에 사용).

한 문장 — 개인정보=양방향, 비밀번호=해시+SALT. 로그인 상태는 토큰(JWT)으로 시작하고, 필요하면 세션을 더한다.

Day 13 · 구글로그인 실습 — GCP·Supabase 연동, JWT의 정체

- 소셜로그인 3단계(페이지 불러오기 → 인증 → 인가)와 세 토큰(비밀 CODE · 구글 전용 · Supabase 전용).
- 연동 = 서로의 값을 맞바꾸기: GCP의 Client ID/Secret ↔ Supabase의 Callback URL. (구현보다 연동 설정이 8할)
- ★ JWT는 암호화가 아니라 인코딩 — 누구나 열어볼 수 있으므로 비밀을 담지 말 것.
- 인가 실물: Network 탭의 Authorization: Bearer <토큰> 헤더. 화면 그림이 아니라 이 토큰이 진짜 신원.

한 문장 — 소셜로그인의 8할은 연동 설정(ID/Secret ↔ Callback URL)이고, JWT는 인코딩이라 누구나 열어본다.

Day 14 · 혹스와 기능 병렬 구현 — 자동 안전장치와 Git 워크트리

- 사람도 AI도 읽는다 → 혹스로 강제: PreToolUse(도구 사용 전 차단, 예: .env 접근 deny) · Stop(작업 종료 전 검사, 하드코딩 감지 → .env.example 분리). 탐지는 스크립트, 판단은 모델.
- /update-config 로 생성, /hooks 로 확인(읽기 전용). ⚠️ .gitignore 의 .env* 한 줄이 .env.example 까지 죽임 → !.env.example 을 뒤에.

- 토큰맥싱 → 토크노믹스: "많이 쓰기"가 목표가 되면 지표가 망가짐(굿하트의 법칙). 정답은 "재보고 정하기".
- 병렬 구현의 난관 = 공통 파일 수정 충돌. 폴더 복사=합치기 불가, Git 워크트리= .git 공유로 합치기 가능. 기능은 의존 관계로 그룹화, 작업 순서 5단계(분리 → 병렬구현 → 통합 → 리팩토링 → 동기화).

한 문장 — 규칙은 문서가 아니라 흑으로 강제하고, 병렬 구현은 폴더 복사가 아니라 Git 워크트리로 통합한다.

Day 15 · 워크트리 병렬 실전과 이미지 프로세스

- 노션 기능을 의존 관계 4그룹(게시글 CRUD · 다크모드 · 사이드바 · 자기소개)으로 워크트리 4개에 배분, /speckit-specify 로 병렬 구현("DB 구조 변경 금지" 같은 제약을 스펙에 명시).
- 확인·커밋 후 "워크트리 모두 통합해줘" → 서버 종료 → diff 파악 → 순차 merge → 재기동 검증. 디버깅: "네가 로그를 남겨줘".
- 이미지 = 픽셀의 조합, 픽셀 = RGB 세 숫자(0~255). 256인 이유 = 8비트 = 2^8 . DB 저장 타입 BLOB이나 직접 저장은 조화를 느리게.
- 비동기 분리 저장: 실물은 Supabase Storage(버킷 · UUID 파일명 · S3 호환), DB엔 다운로드 주소만. 스토리지경로= .env, 파일경로=DB.

한 문장 — 기능은 의존 관계로 묶어 워크트리로 병렬 통합하고, 이미지는 Storage에 주소만 DB에(비동기 분리).

Day 16 · 배포 환경과 모니터링, 그리고 총정리

- 배포 4단계: 로컬 → 개발 → 스테이징 → 프로덕션(단계별 도메인 · .env · 회원 데이터 분리). 1인 개발은 로컬 +프로덕션만으로 시작(.env.prod / .env.local).
- Git·Vercel 자동 배포: "빌드 → 커밋 → 푸쉬" → GitHub↔Vercel 자동 감지·배포. ⚠️ .env 는 GitHub이 아니라 Vercel에 직접 입력.
- 배포 후 소셜로그인이 막히면 → Supabase Site URL · Redirect URL을 배포 주소로 갱신(프로덕션·로컬).

(심화) 커스텀 도메인 연결 — DNS 레코드 · xxx.vercel.app 대신 내 도메인(예: eqment.store)을 붙이려면, 도메인 관리 콘솔(가비아·Cloudflare 등)에서 DNS 레코드를 직접 지정한다

레코드	연결 대상	언제 쓰나
A	도메인 → IP 주소	루트 도메인(@ , mynotion.com)을 IP에 직접 꽂을 때
CNAME	도메인 → 다른 도메인	서브도메인(www. · dev. · staging.)을 Vercel 주소로 넘길 때
TXT	텍스트 정보	소유권 검증 — "이 도메인이 내 것"임을 플랫폼에 증명
MX / SRV	메일 / 서비스	메일 수신 등 부가 용도

- 레코드 한 줄 = **호스트**(@ =루트, `www` · `dev` =서브도메인) + **값/위치**(IP 또는 대상 도메인) + **TTL**(캐시 유지 시간, 예: 600초).
- 💡 **A와 CNAME이 따로 있는 이유** — 루트 도메인(@)에는 CNAME을 쓸 수 없어 **A(IP 직접)**, 서브도메인은 대상이 바뀌어도 따라가도록 **CNAME(도메인 참조)**. 그래서 배포 4단계의 `staging.` · `dev.` 서브도메인 분리도 CNAME으로 한다.
- ⚠️ 바뀌도 **TTL만큼은 옛 값이 캐시에 남아** 즉시 반영되지 않을 수 있다(전파 대기).
- ⚠️ 커스텀 도메인을 붙였다면 **Supabase의 Site URL·Redirect URL도 새 도메인 기준으로 다시 갱신**해야 로그인이 이어진다.
- **모니터링: 로그**(요청 기록) vs **메트릭**(시간에 따라 변하는 수치). **Vercel 에이전트**(Chat·Code Reviews·Investigations)에 "프로젝트/기간/증상/결과"를 담아 질문 — "이 도메인 DNS/TLS 문제가 뭔지 진단해줘"처럼 도메인 문제도 맡길 수 있다.

한 문장 — 빌드→커밋→푸쉬면 Vercel이 자동 배포한다. `.env` 는 Vercel에 직접 넣고, 배포 후 소셜로그인 URL을 갱신하라.

전권 관통 원칙

1. **역할 분리형 + 핸드오프** — 기획(PM-Skills) → 디자인(클로드 디자인) → 개발(하네스)로 잇는다.
2. **메타프롬프트·CLAUDE.md로 결정론화** — 결과물이 아니라 프롬프트를 만들고, 규칙은 CLAUDE.md에 못 박는다.
3. **정규화와 UUID** — 중복은 PK/FK로 분리, 식별자는 UUID(쿠팡 유출 교훈).
4. **하네스로 AI를 통제** — TDD/SDD로 명세 먼저·구현은 그대로, 설계 후 컨텍스트 초기화.
5. **CORS는 프록시로** — 브라우저가 막으면 내 서버(프록시)가 대신 요청.
6. **개인정보=양방향, 비밀번호=해시+SALT** — 로그인 상태는 토큰(JWT)으로 시작.
7. **구조로 강제하라** — 규칙은 문서가 아니라 흑으로, 병렬은 워크트리로, 비밀은 저장소 밖에.

전권 핵심 용어 사전

용어	한 줄 정의
통합형 / 분리형	한 번에 뽑기(빠름) / 기획→디자인→개발 단계(일관성)
핸드오프 / 로-파이·하이-파이	산출물을 다음 역할에 넘김 / UX 뼈대·UI 완성

전권 핵심 용어 사전

용어	한 줄 정의
PM-Skills / 기획 7단계	기획용 슬래시 커맨드 / 인터뷰→요약→...→PRD
<u>DESIGN.md</u> / 메타프롬프트	디자인 단일 원본 / 결과물을 만들 프롬프트를 먼저
확률적 / 결정론적	지킬 수도 안 지킬 수도 / 규칙이 예외 없이 우선
정규화 / PK·FK·Join	중복 분리 / 식별·참조·재결합
1:1·1:N·N:M / 중간 테이블	관계 3유형 / N:M을 두 개 1:N으로 분해
UUID / auto_increment	랜덤(보안) / 순차 정수(예측 가능, 쿠팡 유출)
LLM 4단계	프롬프트→컨텍스트→하네스→루프
TDD / SDD	테스트 먼저(회귀 방지) / 명세 먼저(6715→399줄)
Superpowers / Speckit	TDD 플러그인 / SDD(/specify ~ /implement)
API 엔드포인트 / JSON	프론트(HTML)·백엔드(JSON) / 실무 API=JSON
GET / POST·PATCH·DELETE	조회 / 등록·수정·삭제(Postman)
상태코드 200·400·500	성공 / 클라이언트 잘못 / 서버 잘못
SOP / CORS / Origin	같은 출처만 / 예외 허용 / 브라우저에만 해당
프록시 서버	브라우저 대신 외부 API를 요청(CORS 우회)
인증 / 인가	증표 받기(1회) / 증표로 증명(매번, header)
양방향 / 단방향(해시)+SALT	개인정보(복호화 O) / 비밀번호(복호화 X)

용어	한 줄 정의
세션 / 토큰(JWT)	세션 테이블(강제 로그아웃 O) / 토큰 내장(효율)
OAuth / 소셜로그인 3토큰	위임 표준 / 비밀 CODE→구글 토큰→Supabase 토큰
PreToolUse / Stop	도구 사용 전 차단 / 종료 전 검사
토큰맥싱 / 토크노믹스	많이 쓰기 / 재보고 정하기(굿하트의 법칙)
폴더 복사 / Git 워크트리	합치기 불가 / .git 공유로 통합 가능
픽셀·RGB / BLOB	사진=RGB(0~255, 256=2 ⁸) / 이미지 DB 타입

비동기 분리 저장	실물은 Storage(UUID)-주소만 DB
배포 4단계 / <code>.env</code> 등록	로컬·개발·스테이징·프로덕션 / Vercel에 직접
DNS 레코드 A / CNAME / TXT	도메인→IP(루트) / 도메인→다른 도메인(서브) / 소유권 검증
호스트 / TTL	@ =루트 · www =서브도메인 / 캐시 유지 시간(전파 대기)
로그 / 메트릭	요청 기록 / 시간에 따라 변하는 수치

Phase 2 졸업 체크리스트

- ☐ 기획(PRD) → 디자인(디자인 시스템·하이파이) → 개발 핸드오프를 한 사람이 이어서 수행했다
- ☐ 미니 노션이 정규화된 UUID 기반 DB(Supabase) 위에서 동작한다
- ☐ 외부 API를 스켈레톤 UI + 프록시 구조로 연동했다
- ☐ 구글로그인이 붙어 있고, 인가(Bearer 토큰)가 작동함을 검증했다
- ☐ 혹(PreToolUse·Stop)이 등록되어 비밀 값 유출을 구조로 막는다
- ☐ 기능 4그룹을 워크트리로 병렬 구현하고 충돌 없이 한 코드베이스로 통합했다
- ☐ 프로필 이미지가 Storage 버킷(UUID 파일명)에 저장되고, DB에는 다운로드 주소만 남는다
- ☐ 미니 노션을 GitHub-Vercel로 배포하고, `.env` 를 Vercel에 직접 넣어 프로덕션·로컬 양쪽에서 소셜로그인이 동작한다
- ☐ 배포된 서비스를 로그·메트릭(또는 Vercel 에이전트)으로 관찰할 수 있다

이전 단계 ← [Phase 1 · 소개 페이지 요약본](#) — 혼자서 만들고 배포하는 기본기(Day 1-5).